

AuthService – Architecture & Topology Diagrams

Generated on 2025-11-05

High-Level Architecture (Mermaid)

```
%% AuthService High-Level Architecture
flowchart LR
    Client[Client / Swagger / Postman]
    API[AuthService.Api (Controllers)]
    App[Application (MediatR CQRS, Validators)]
    Abs[Abstractions (Interfaces)]
    Dom[Domain (Entities, Value Objects)]
    Infra[Infrastructure (EF/Identity/JWT/Repos)]
    Write[(SQL Server - WriteDbContext)]
    Read[(PostgreSQL - ReadDbContext)]
    Aspire[(Aspire AppHost - optional)]

    Client --> API
    API --> App
    App --> Abs
    App --> Dom
    App --> Infra
    Infra --> Dom
    Infra --> Write
    Infra --> Read
    Aspire -. optional orchestration .-> API
    Aspire -. optional orchestration .-> Write
    Aspire -. optional orchestration .-> Read
```

Authentication Flows (Mermaid)

```
%% Authentication Flows
sequenceDiagram
    participant U as User
    participant API as API
    participant App as Application (MediatR)
    participant Id as Identity (UserManager/SignInManager)
    participant JWT as JwtTokenService
    participant SQL as WriteDbContext
    participant PG as ReadDbContext

    U->>API: POST /api/auth/register
    API->>App: RegisterCommand
    App->>Id: Create user + add role
    Id->>SQL: Save user
    App-->>U: Confirmation link (log)

    U->>API: GET /api/auth/register/confirm?userId&token
    API->>App: ConfirmCommand
    App->>Id: Confirm Email
    App-->>U: OK

    U->>API: POST /api/auth/login
    API->>App: LoginCommand
    App->>Id: PasswordSignInAsync
    App->>JWT: CreateAccessToken + RefreshToken
    App-->>U: tokens

    U->>API: GET /api/users/me (Bearer)
    API->>App: GetProfileQuery
    App->>Id: Load user + roles
    App-->>U: Profile

    U->>API: POST /api/users/addresses (Bearer)
    API->>App: CreateAddressCommand
    App->>SQL: Insert Address
```

```
App-->>U: Created
U-->>API: GET /api/users/addresses (Bearer)
API-->>App: GetAddressesQuery
App-->>PG: Query Addresses
App-->>U: List
```

Compose + Aspire Topology (Mermaid)

```
%% Docker Compose + Aspire Resource Topology
graph TD
    subgraph Compose
        A[auth_api] -->|depends_on| B[(sqlserver)]
        A -->|depends_on| C[(postgres)]
    end

    subgraph Aspire (optional)
        H[AppHost]
        H --> A
        H --> B
        H --> C
    end

    A -->|:8080| User[Client]
    B -->|:1433| DBA1[SSMS]
    C -->|:5432| DBA2[psql]
```